
THE HP PROTEIN-FOLDING PROBLEM: MODERN CONSTRAINT PROGRAMMING AND REINFORCEMENT LEARNING APPROACHES

A PREPRINT

 **Pranav Ramanathan**

School of Mathematical Sciences
Queen Mary University of London
p.ramanathan@se23.qmul.ac.uk

 **Thomas Prellberg**

School of Mathematical Sciences
Queen Mary University of London
t.prellberg@qmul.ac.uk

May 6, 2026

ABSTRACT

This paper presents a modern approach to constraint programming for the HP protein folding problem using Google OR-Tools CP-SAT, and compares it with Reinforcement Learning (RL). The HP model is designed to model a protein as a sequence of H (hydrophobic) and P (polar) amino acids on a simple cubic lattice with the interactions being chosen to mimic an aqueous solvent. Minimising its energy is a classic NP-complete problem in computational physics and combinatorial optimization, that has been studied extensively. CP-SAT is an open-source constraint programming solver that systematically explores the solution space through constraint satisfaction, dynamically augmenting itself with learned clauses. RL folds proteins by interacting with the environment and optimizing a policy through trial-and-error. On the shorter benchmark sequences tested, CP-SAT matches the best-known energy levels and provides certificates of optimality where it terminates with OPTIMAL status; on the longer sequences both methods remain below the best-known energies under the tested budgets.

Keywords HP model · protein folding · constraint programming · CP-SAT · reinforcement learning · lattice models

1 Introduction

Dill introduced the Hydrophobic-Polar (HP) lattice model in 1985 [1]. This model collapses the twenty amino acid types into two classes: hydrophobic residues (H), which tend to avoid water, and polar residues (P), which interact more favourably with it. A sequence is then embedded as a self-avoiding walk on a lattice, with each residue occupying a distinct site and successive residues constrained to lie on adjacent lattice points [2]. Within this discrete setting, the objective is driven by hydrophobic contacts. In the simpler version of the model, the energy is defined by the number of non-consecutive H-H contacts, i.e., pairs of hydrophobic residues that are neighbours on the lattice but not adjacent along the chain. In effect, the optimisation target is to find an embedding that promotes hydrophobic clustering while respecting the self-avoidance and adjacency constraints.

Even under this simplified lattice formulation, HP folding remains computationally demanding. *Crescenzi et al.* proved in 1998 that the optimisation problem is NP-complete [3], so one should not expect a method that scales efficiently across all instances as the sequence length increases. The practical consequences show up fairly quickly: by the time sequences are in the 30 to 50 residue range, the set of valid self-avoiding conformations is already large enough that exhaustive enumeration stops being a useful point of comparison. That is part of the reason the HP model is used so often in combinatorial optimisation research: it is clean to specify, hard to solve, and easy to evaluate consistently across algorithms.

Given that hardness, much of the earlier computational literature relied on heuristics rather than exact optimisation. Simulated annealing became a common choice from the 1990s onward because it provides a straightforward mechanism for escaping local minima by occasionally accepting unfavourable moves early in the search [4]. Genetic algorithms offered a different style of exploration, maintaining a population of candidate folds and gradually improving it through

recombination and mutation operators [5]. Tabu search was also explored, discouraging cycling by remembering recent states. Ant colony optimisation took yet another approach, biasing construction using information carried over from successful searches [6, 7].

Constraint programming (CP) offers an alternative to heuristic search for HP folding. Chain connectivity, self-avoidance, and contact formation can all be encoded directly as folding constraints and then pruned through propagation [8]. Earlier CP tools, such as CPSP-tools, successfully tackled HP folding using specialised H-core databases and constraint satisfaction techniques [9]. Google OR-Tools CP-SAT extends this approach by combining it with on-the-fly SAT-style clause learning and linear programming relaxation [10]. This can provide bounds on solution quality and, in favourable cases, certificates of optimality [11]. Large Neighbourhood Search is also supported, allowing solutions to be improved iteratively [12]. Earlier domain-specific tools such as CPSP-tools [9] solve HP folding by enumerating a precomputed database of optimal H-cores (connected lattice subgraphs of hydrophobic residues), then asking a constraint solver only whether the remaining polar residues can be threaded around each candidate core; this offline precomputation cannot be reused across lattice types or model variants. Our CP-SAT formulation instead encodes the full folding problem directly, trading some HP-specific power for portability, reproducibility, and direct comparability against a learning-based baseline on the same benchmark family.

While constraint programming guarantees optimality when feasible, its exponential scaling limits practical applicability to shorter sequences. Reinforcement learning has recently emerged as an alternative approach for HP folding. An attention-based deep Q-learning approach has recently been applied to the three-dimensional HP model [13]. Folding was treated as a sequential decision problem, where agents can learn folding strategies through trial-and-error without explicitly encoding the problem constraints [14]. Whether this reinforcement learning method can compete with exact solver approaches like CP-SAT remains unknown.

This paper presents a modern approach to constraint programming for the 3D HP protein folding problem using Google OR-Tools CP-SAT, comparing it against the recent attention-based deep Q-learning method of [13]. To enable a like-for-like comparison, we additionally rerun the RL baseline on hardware comparable to its published evaluation, so that both methods are exercised under the same operating conditions. Section 2 describes the implementation and training procedures for each method. Section 3 presents comparative results and scaling behaviour. Section 4 discusses algorithmic trade-offs, failure modes, and future research directions.

2 Methodology

2.1 Problem Formulation

The HP protein folding problem on a three-dimensional lattice is defined as follows. A protein sequence is represented as a string:

$$S = s_1 s_2 \dots s_n, \quad (1)$$

where each residue s_i is classified as either hydrophobic (H) or polar (P). A conformation C is a self-avoiding walk on the three-dimensional integer lattice $\mathbb{Z}^3$, meaning each residue is placed at a distinct position:

$$c_i = (x_i, y_i, z_i) \in \mathbb{Z}^3, \quad (2)$$

and no two residues occupy the same lattice point. The adjacency constraint requires that consecutive residues along the chain are placed at Manhattan distance 1:

$$\|c_i - c_{i+1}\|_1 = 1 \quad \forall i \in \{1, \dots, n-1\}. \quad (3)$$

where $\|v\|_1 = |x| + |y| + |z|$ for a vector $v = (x, y, z)$.

The energy of a conformation is determined by the number of non-consecutive H-H contacts. A contact occurs when two hydrophobic residues are neighbours on the lattice but not adjacent along the sequence. Formally, the energy function is defined as

$$E(C) = -|\{(i, j) : s_i = s_j = \text{H}, |i - j| > 1, \|c_i - c_j\|_1 = 1\}|, \quad (4)$$

where the set counts all pairs of non-consecutive H-residues that are lattice-adjacent. The objective is to minimise $E(C)$, which is equivalent to maximising the number of favourable hydrophobic contacts. This formulation reflects the simplified physical principle that hydrophobic residues prefer to cluster together, shielding themselves from the surrounding aqueous environment. An illustration of this energy function is provided in Figure 1.

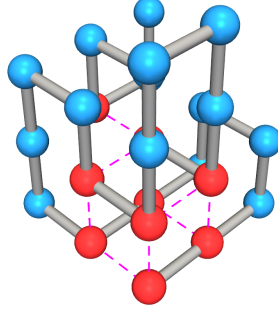


Figure 1: Illustration of the 3D HP lattice model showing a short self-avoiding walk. Hydrophobic residues (H) are shown in red, polar residues (P) in blue. Dotted magenta lines indicate valid non-consecutive H-H contacts that contribute to the energy function; this conformation has energy $-9\epsilon_{HH}$.

We evaluate both methods on a set of benchmark sequences drawn from the literature. Table 1 lists the sequences used in this study, identified by their standard names (e.g., 3d1, 3d2, ..., 3d8). The sequences span a range of lengths from $n = 20$ to $n = 58$ residues and include both their H/P patterns and best-known energy values from prior work [13].

Table 1: Benchmark HP sequences used in this study. Best-known energies are taken from [13].

Seq.	Length	Sequence	Best-known Energy
3d1	20	$(HP)^2PH(HP)^2(PH)^2HP(PH)^2$	-11
3d2	24	$H^2P^2(HP^2)^6H^2$	-13
3d3	25	$P^2HP^2(H^2P^4)^3H^2$	-9
3d4	36	$P(P^2H^2)^2P^5H^5(H^2P^2)^2P^2H(HP^2)^2$	-18
3d5	46	$P^2H^3PH^3P^3HPH^2PH^2P^2HPH^4$	-35
		$PHP^2H^5PHPH^2P^2H^2P$	
3d6	48	$P^2H(P^2H^2)^2P^5H^{10}P^6$	-31
		$(H^2P^2)^2HP^2H^5$	
3d7	50	$H^2(PH)^3PH^4PH(P^3H)^2P^4$	-34
		$(HP^3)^2HHP^4(PH)^3PH^2$	
3d8	58	$PH(PH^3)^2P(PH^2PH)^2H(HP)^3$	-44
		$(H^2P^2H)^2PHP^4(H(P^2H)^2)^2$	

2.2 CP-SAT Approach

This section describes our constraint programming implementation using Google OR-Tools CP-SAT solver [11, 10]. We formulate the 3D HP folding problem as a constraint satisfaction problem, encoding the self-avoiding walk and contact constraints algebraically to maximise H-H contacts.

2.2.1 Decision Variables

Our CP-SAT formulation uses one-hot encoding to represent where residues sit on the 3D cubic lattice. In one-hot encoding, each position is indicated by a binary vector where exactly one element is set to 1 (indicating the residue occupies that position) and all others are 0. For each residue $i \in \{0, 1, \dots, n-1\}$ and each cell $c = (x, y, z)$ in the grid:

$$\mathcal{G} = \{(x, y, z) : x, y, z \in \{0, 1, \dots, L-1\}\}, \quad (5)$$

we create a binary decision variable:

$$x_{i,c} \in \{0, 1\}, \quad (6)$$

where $x_{i,c} = 1$ means residue i occupies position c , and $x_{i,c} = 0$ otherwise. The grid dimension L needs to provide enough space for sequences of length n , typically satisfying $L \geq \lceil n^{1/3} \rceil + 2$.

For each pair of non-consecutive hydrophobic residues (i, j) with $|i - j| > 1$ and $S_i = S_j = \text{H}$, we introduce contact indicators:

$$c_{i,j} \in \{0, 1\}, \quad (7)$$

where $c_{i,j} = 1$ if residues i and j occupy adjacent cells at Manhattan distance 1, and $c_{i,j} = 0$ otherwise.

2.2.2 Core Constraints

The CP-SAT model enforces four main constraint types:

Placement constraint. Each residue must sit in exactly one cell:

$$\sum_{c \in \mathcal{G}} x_{i,c} = 1 \quad \forall i \in \{0, \dots, n-1\}. \quad (8)$$

Self-avoidance constraint. Each cell can hold at most one residue:

$$\sum_{i=0}^{n-1} x_{i,c} \leq 1 \quad \forall c \in \mathcal{G}. \quad (9)$$

Adjacency constraint. Consecutive residues in the sequence must occupy neighbouring cells on the lattice, creating the self-avoiding walk. For each consecutive pair $(i, i+1)$, if residue i sits in cell c , then residue $i+1$ must occupy one of the six adjacent cells:

$$\mathcal{N}(c) = \{c' \in \mathcal{G} : \|c - c'\|_1 = 1\}. \quad (10)$$

We encode this using logical implications:

$$x_{i,c} = 1 \implies \sum_{c' \in \mathcal{N}(c)} x_{i+1,c'} = 1 \quad \forall i \in \{0, \dots, n-2\}, \forall c \in \mathcal{G}. \quad (11)$$

Contact linking constraint. The contact indicators $c_{i,j}$ are linked to placement variables through reified constraints (constraints represented as boolean variables that can be true or false) that ensure $c_{i,j} = 1$ exactly when residues i and j occupy adjacent cells, while excluding consecutive backbone edges.

2.2.3 Symmetry Breaking

The cubic lattice has 48 symmetries from rotations and reflections. To cut down the search space and speed up solving, we fix the first two residues at the grid center:

$$x_{0,(\lfloor L/2 \rfloor, \lfloor L/2 \rfloor, \lfloor L/2 \rfloor)} = 1, \quad (12)$$

$$x_{1,(\lfloor L/2 \rfloor, \lfloor L/2 \rfloor, \lfloor L/2 \rfloor - 1)} = 1. \quad (13)$$

This pins residue 0 at the center and residue 1 along the negative z -axis, which canonicalises the representation by providing a unique representative conformation from each equivalence class of symmetry-related solutions. This approach shrinks the search space by a factor of 48 without losing any optimal solutions.

2.2.4 Objective Function

The goal is to maximise the total number of non-consecutive H-H contacts. Let $\mathcal{H} = \{i : s_i = \text{H}\}$ denote the set of hydrophobic residue indices. The objective is:

$$\text{maximise} \quad \sum_{(i,j) \in \mathcal{C}} c_{i,j}, \quad (14)$$

where $\mathcal{C} = \{(i, j) : i, j \in \mathcal{H}, i < j, |i - j| > 1\}$ is the set of potential non-consecutive H-H contact pairs. Since the energy function is $E(C) = -\varepsilon_{HH} \cdot \sum_{(i,j) \in \mathcal{C}} c_{i,j}$, maximising contacts is equivalent to minimising energy.

2.2.5 Solver Configuration

We use Google OR-Tools CP-SAT version 9.11 with this setup for each sequence:

- **Optimisation phase:** 560 seconds (20% of time budget) for finding the best grid size and tuning parameters across 5 random seeds
- **Final solve phase:** 2240 seconds (80% of time budget) with optimised settings
- **Parallelisation:** 8 worker threads running parallel search
- **Total time budget:** 2800 seconds (roughly 47 minutes per sequence)

We determine grid size L during the optimisation phase by testing values from $\lceil n^{1/3} \rceil + 2$ up to this value plus 3, picking the smallest grid that gives good solutions. The solver uses branch-and-bound search (a divide-and-conquer method that recursively splits the solution space while pruning branches proven suboptimal) with constraint propagation, conflict-driven clause learning, and automatic search strategies. When the solver finishes with OPTIMAL status, the solution is provably the best possible. When it stops with FEASIBLE status, the solution is valid but we can't guarantee it's optimal.

2.3 Attention-based DQN Baseline

This section describes the attention-based deep Q-learning method of [13], which we use as the learning-based baseline. The method trains an agent to place residues sequentially on the 3D lattice, maximising H-H contacts through trial-and-error interaction with the environment, with action-value estimates produced by an attention-based Q-network. We use the authors' publicly available implementation and rerun the training protocol on hardware comparable to its published evaluation, so that the comparison with CP-SAT is exercised under similar operating conditions. Earlier project notes used the working label *LSTM-A*; in the final comparison we describe the baseline according to the inspected implementation and target paper, namely an attention-based DQN.

2.3.1 Markov Decision Process Formulation

The 3D HP folding problem is formulated as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

The **state space** $\mathcal{S}$ consists of one-hot encoded vectors representing the folding trajectory. Each state is represented as a tensor of shape $(n, 8, 1)$, where n is the sequence length. At each position in the sequence, the 8-dimensional vector encodes:

$$s_i = [\text{ND}, \text{F}, \text{L}, \text{R}, \text{U}, \text{D}, \text{H}, \text{P}], \quad (15)$$

where the first six elements represent the available actions (No Decision, Forward, Left, Right, Up, Down) and the last two elements indicate the amino acid type (Hydrophobic or Polar). Importantly, the network does not directly observe 3D coordinates; instead, it infers spatial configurations from the sequence of actions taken.

The **action space** $\mathcal{A}$ consists of five discrete relative placement directions:

$$\mathcal{A} = \{\text{Forward}, \text{Left}, \text{Right}, \text{Up}, \text{Down}\}. \quad (16)$$

Each action specifies where to place the next residue relative to the current chain orientation. Backward moves are excluded since backtracking would violate the self-avoiding walk constraint.

The **transition dynamics** are deterministic:

$$P(s_{t+1} | s_t, a_t) : \text{deterministic}, \quad (17)$$

where action a_t determines the placement of the next residue based on the current chain orientation. Episodes terminate successfully when all n residues are placed without collision, or terminate early if the agent reaches a configuration where no valid placements remain.

The **reward function** $R(s_t, a_t)$ is sparse, providing feedback only upon episode termination:

$$R(s_t, a_t) = \begin{cases} \sum_{(i,j)} c_{i,j} & \text{if all residues placed successfully} \\ 0 & \text{otherwise} \end{cases}, \quad (18)$$

where $c_{i,j} = 1$ indicates a valid H-H contact between non-consecutive hydrophobic residues i and j at Manhattan distance 1. This reward equals the negative energy $-E(C)$ from the problem formulation.

2.3.2 Environment Implementation

The reference implementation [13] uses a custom Gymnasium environment with the following features:

Bounded coordinate system. Residue positions are constrained to lie within a cubic bounding box:

$$[-r, r]^3 \quad \text{where} \quad r = \lfloor n/2 \rfloor. \quad (19)$$

This prevents unbounded exploration and focuses search on compact conformations.

Symmetry breaking. To reduce the search space by a factor of 48 (24 rotational symmetries $\times$ 2 reflections), the environment enforces two constraints: the first non-forward turn must be to the right, and the first vertical deviation must be upward. This canonicalises equivalent conformations under lattice symmetries, providing a unique representative from each symmetry class.

Trap detection. The environment identifies dead-end configurations where no valid actions remain for the next residue placement. When trapped, the episode terminates immediately with zero reward, providing an implicit penalty that discourages exploration of unproductive regions.

Action masking. At each step, the environment computes a boolean mask:

$$m \in \{0, 1\}^{|\mathcal{A}|}, \quad (20)$$

indicating which actions satisfy three criteria: they do not violate the bounded region, they do not cause collisions with existing residues, and they do not lead immediately to a trapped state. This mask is used during action selection to restrict the agent’s choices to feasible actions only.

2.3.3 Attention-based Q-Network

The action-value function $Q(s, a)$ is approximated by an attention-based Q-network as described in [13]. The $(n, 8, 1)$ state tensor is embedded and processed by a multi-head self-attention encoder, which allows each position to attend to all previously placed residues and captures both local sequential patterns and long-range dependencies in the partial conformation. The encoder output at the final position is mapped through a fully connected layer to Q-values for the five placement actions. Architecture-level hyperparameters (embedding dimension, number of heads, number of encoder layers) follow the values reported in [13] and are summarised, alongside training settings, in Table 2.

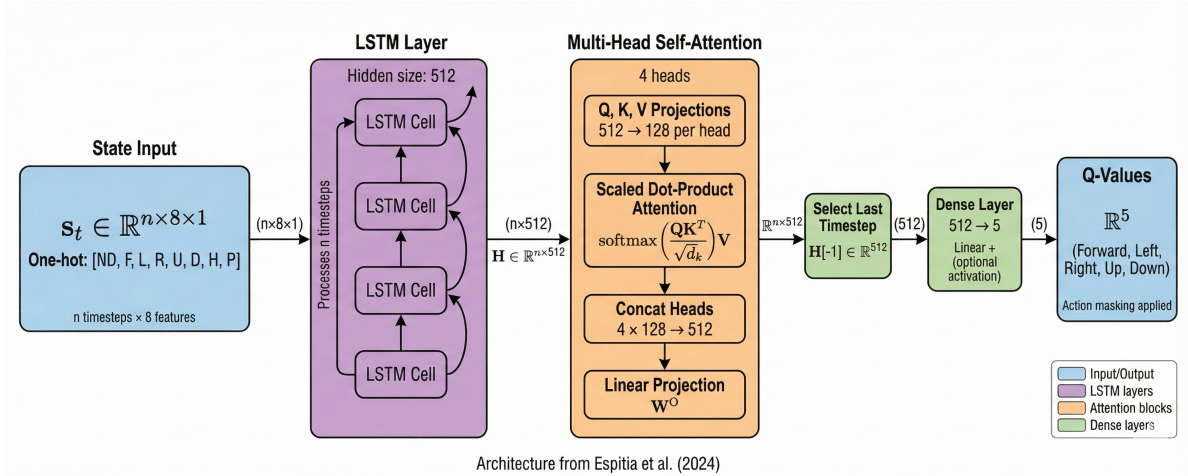


Figure 2: Attention-based Q-network used in the baseline [13]. The $(n, 8, 1)$ state input is embedded and processed by a multi-head self-attention encoder, and the encoder output at the final position is mapped to Q-values for the five placement actions through a dense layer.

2.3.4 Training Protocol

The training procedure in [13] uses Deep Q-Learning with several standard stabilisation techniques: a Double DQN target with a periodically synchronised target network to mitigate overestimation bias, prioritised experience replay

(with annealed importance-sampling correction) so that high-TD-error transitions receive more learning signal, and an ε -greedy exploration policy restricted to the valid actions indicated by the environment’s action mask. Full equations for these techniques follow standard formulations and are not reproduced here; we refer the reader to [13] for the exact specification.

Hyperparameters and training details. Following [13], separate models are trained for each benchmark sequence. Table 2 summarises the training configuration used.

Table 2: Attention-based DQN training hyperparameters reported by [13].

Parameter	Value
<i>Network Architecture</i>	
Embedding dimension d_{model}	512
Attention heads	4
<i>Training Configuration</i>	
Optimiser	Adam
Learning rate	1.0×10^{-3}
Discount factor γ	0.98
Batch size	16 (short seq.), 32 (long seq.)
Training episodes	200K–250K (short), 500K–750K (long)
<i>Stabilisation Techniques</i>	
Target network update	Every 1,000 steps
PER α (prioritisation)	0.6
PER β (importance sampling)	$0.4 \rightarrow 1.0$ (annealed)
ε -greedy decay	$\max(0.01, 1.0 \cdot 0.9995^t)$

3 Experiments and Results

3.1 Experimental Setup

All experiments were conducted on an Apple MacBook Pro with M2 Max chip and 32 GB unified memory. The CP-SAT solver was implemented using Google OR-Tools version 9.11, while the attention-based DQN implementation was obtained from [13] and executed locally. Both methods were evaluated on the standard 3D HP benchmark sequences (3d1–3d8) reported in [13], with best-known energy values serving as optimality targets.

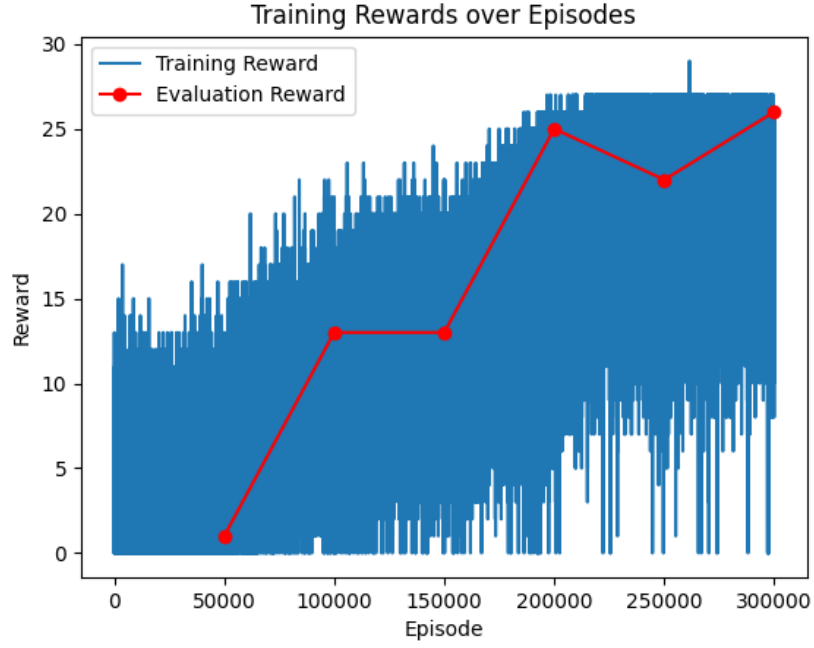
3.2 CP-SAT Experiments

For CP-SAT, we allocated a total computational budget of 7 hours distributed across all 8 sequences (3d1–3d8), yielding approximately 47 minutes per sequence. Each sequence underwent a two-phase optimisation: 20% of the budget (9 minutes, 20 seconds) for grid size exploration and parameter tuning across 5 random seeds, and 80% of the budget (37 minutes, 20 seconds) for final optimisation with the best-discovered settings. The solver used 8 parallel worker threads throughout. All CP-SAT results reported in this chapter were obtained from our own experimental runs.

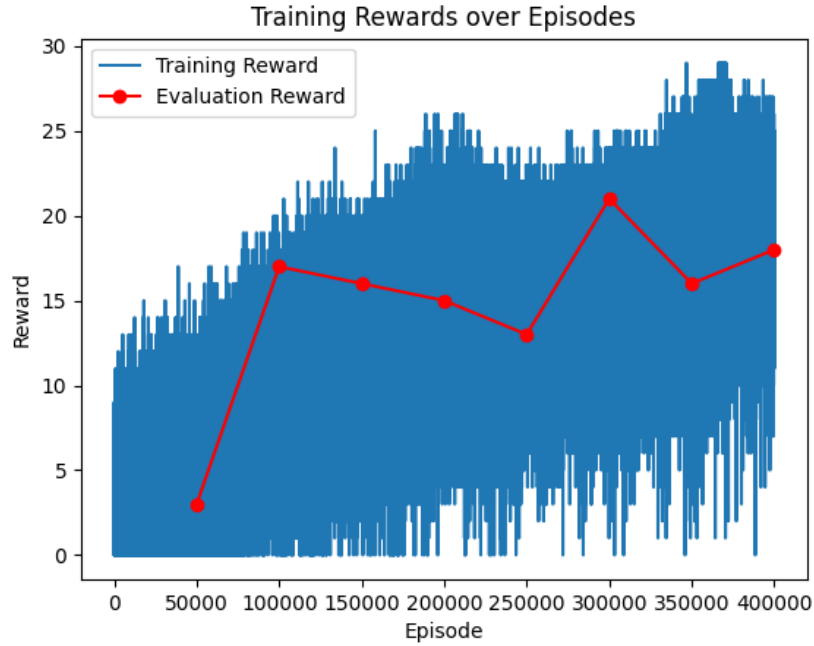
3.3 Attention DQN Experiments

For the attention-based DQN agent, we replicated the training protocol described in Section 2.3 on our local machine using the PyTorch MPS backend (M2 Max). Single-seed 200,000-episode runs were completed for sequences 3d1, 3d2, and 3d3; runs for 3d4 are in progress, and runs for 3d5–3d8 remain pending due to the substantial wall-clock cost of the published protocol.

We therefore report a combination of results: locally reproduced 200,000-episode best-evaluation energies for 3d1–3d3, with published reference values from [13] used for 3d4–3d8 until local reproduction completes. Figure 3 shows representative training dynamics from [13] for sequences of varying lengths, illustrating how training complexity scales with sequence size.



(a) Length 48 (3d6)



(b) Length 50 (3d7)

Figure 3: Attention DQN training convergence curves from [13] for sequences of varying lengths. Blue areas show training rewards, red lines show evaluation performance.

All results tables in this chapter clearly indicate the source of each result, distinguishing between our experimental runs and values taken from the original paper.

3.4 Solution Quality and Comparative Analysis

Table 3 presents the energy values achieved by both methods on the 3D HP benchmark sequences. The energy is defined as the negative count of non-consecutive H-H contacts, so more negative values indicate better (more stable) conformations.

Table 3: Solution quality on 3D HP benchmark sequences. Energy is the negative count of H-H contacts; lower (more negative) values are better. CP-SAT values are from this work. RL Source distinguishes locally reproduced 200,000-episode single-seed values (this work) from published reference values in [13].

Seq.	Len	Best Known	CP-SAT	RL	RL Source
3d1	20	−11	−11	−10	local 200k
3d2	24	−13	−13	−9	local 200k
3d3	25	−9	−9	−9	local 200k
3d4	36	−18	−18	−18	published [13]
3d5	46	−35	−32	−33	published [13]
3d6	48	−31	−29	−30	published [13]
3d7	50	−34	−30	−29	published [13]
3d8	58	−44	−40	−40	published [13]

Table 4: Specialised exact reference (CPSP-tools / HPstruct) on the same benchmark family. CPSP-tools enumerates a precomputed cubic-lattice H-core database offline and uses Gecode at run time to verify that the chain can be threaded around each candidate core; results are reported here as a calibration anchor for the general-purpose CP-SAT formulation rather than as a like-for-like comparison.

Seq.	Len	Best Known	CPSP/HPstruct	Time (s)	CPSP status
3d1	20	−11	−11	0.2	proven optimal
3d2	24	−13	−13	0.2	proven optimal
3d3	25	−9	−9	0.2	proven optimal
3d4	36	−18	−18	0.2	proven optimal
3d5	46	−35	−35	0.4	proven optimal
3d6	48	−31	−31	0.3	proven optimal
3d7	50	−34	−34	13.6	proven optimal
3d8	58	−44	—	1.2	H-core DB coverage limit

CPSP-tools attains the best-known energies on every sequence within its database coverage (3d1–3d7) in seconds. This is consistent with the shape of the method: the offline H-core enumeration has already pre-pruned the search to a small set of candidate cores per energy level, and the run-time work reduces to a sequence of small constraint-satisfaction queries. The CP-SAT gap on 3d5–3d7 therefore reflects the cost of a general-purpose constraint formulation that does not encode H-core structure, rather than a fundamental limit of constraint search on this benchmark family. For sequence 3d8 the bundled cubic H-core database does not contain a core large enough to accommodate the sequence’s H-count; HPstruct terminates cleanly without a structure, leaving CP-SAT and the attention-DQN baseline as the only methods to report a value on this instance. The same database also cannot be reused if the lattice or model is changed, so the comparison should be read as a calibration of CP-SAT on the cubic HP family rather than as a verdict on constraint search overall.

CP-SAT matches the best-known energy values on 3d1–3d4 (lengths 20–36) and terminates with OPTIMAL status on these instances, providing certificates of optimality through exhaustive branch-and-bound search within the allocated time budget. The published attention DQN reference also matches best-known on 3d1–3d4. Our local 200,000-episode reruns of the attention DQN match best-known on 3d3 but fall short on 3d1 (−10 versus −11) and 3d2 (−9 versus −13) under a single seed and the tested budget, suggesting the reproduction settings would benefit from additional seeds or a larger compute envelope.

On longer sequences (3d5–3d8, lengths 46–58), neither method reaches the best-known values. CP-SAT maintains gaps of 2–4 contacts from the literature values and terminates with FEASIBLE status, unable to prove optimality within the time limit as the exponential search space growth overwhelms the constraint propagation and symmetry-breaking techniques. The published attention DQN reference shows similar performance with gaps of 2–5 contacts: −33 on 3d5 (within 2 of best-known), −30 on 3d6 (within 1), −29 on 3d7 (5 away), and −40 on 3d8. Notably, both methods

converge to identical energy values on the longest sequence 3d8 (length 58), suggesting they face similar fundamental challenges in scaling to longer sequences.

Figure 4 visualises these results, clearly showing the transition from optimal performance on shorter sequences to increasing gaps on longer ones.

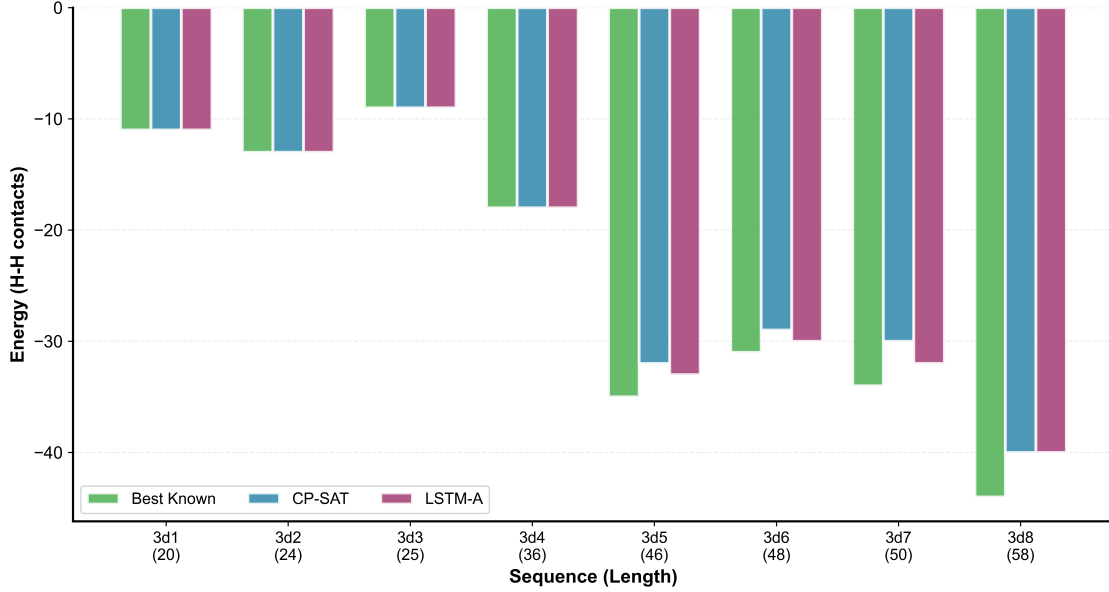
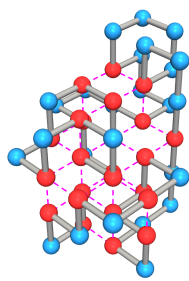
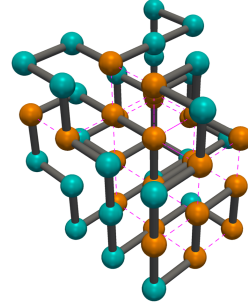


Figure 4: Solution quality comparison across benchmark sequences. Both methods achieve optimal solutions for sequences up to length 36 (3d1–3d4). Performance degrades gracefully on longer sequences, with gaps of 2–5 contacts from best-known values.

Figure 5 illustrates representative conformations for sequence 3d7 (length 50), where CP-SAT achieved -30 contacts and the published attention DQN baseline reports -29 contacts.



(a) CP-SAT solution: Energy = -30 contacts



(b) Attention DQN baseline solution: Energy = -29 contacts

Figure 5: Conformations for sequence 3d7 (length 50). Panel a: CP-SAT achieves -30 contacts (red = H, blue = P). Panel b: published attention DQN baseline achieves -29 contacts (orange = H, cyan = P). Magenta lines mark non-consecutive H-H contacts.

4 Discussion

This study compared constraint programming (CP-SAT) against an attention-based DQN reinforcement-learning baseline for the 3D HP protein folding problem, addressing whether systematic search with guarantees or learning-based exploration better handles this NP-hard combinatorial optimisation task.

On sequences up to length 36, CP-SAT and the published attention DQN baseline both reach the best-known energies, with CP-SAT additionally providing optimality certificates through exhaustive search. The attention-based architecture captures geometric patterns for maximising hydrophobic contacts, indicating that neural networks can learn useful structure of the conformational landscape. Performance diverged on longer sequences (46–58 residues), where both methods fell 2–5 contacts short of best-known values. CP-SAT’s gaps reflect exponential search-space growth overwhelming constraint propagation, while the RL baseline’s gaps suggest learned policies struggle to generalise beyond training distributions. Their convergence to identical energies on the longest sequence (3d8: both achieving –40) indicates they encounter similar local optima despite completely different search strategies.

CP-SAT provides optimality guarantees when tractable, making it ideal for benchmarking and establishing ground truth. The solver’s deterministic behaviour ensures reproducibility and enables rigorous analysis through statistics. However, NP-hardness fundamentally limits scalability, and even sophisticated pruning eventually fails as the search space grows exponentially. The attention-based DQN learns placement policies from experience, potentially enabling faster solutions on similar sequences through transfer learning, and naturally models long-range relationships through self-attention.

Limitations. This study has several limitations. First, CP-SAT here is a modern implementation of constraint programming for HP folding rather than a new constraint-programming theory. Second, CP-SAT wall-clock budgets and RL training episodes are not directly equivalent measures of compute, so the comparison should be interpreted as an empirical benchmark rather than a strict speed contest. Third, some RL values are taken from the published reference [13] until local reproduction runs are complete; our local 200,000-episode single-seed reruns currently match the published reference only on 3d3 and remain below it on 3d1 and 3d2 under the tested budget, and runs for 3d4–3d8 are still in progress. Fourth, the HP model is intentionally simplified and should be viewed as a computational benchmark rather than a realistic all-atom protein-folding model. Fifth, CPSP-tools is not an apples-to-apples baseline for CP-SAT. Its solution speed reflects an offline H-core precomputation, and it returns no result for sequences whose H-count exceeds the bundled database (as observed on 3d8) or for lattice variants other than 3D cubic.

Future work should explore hybrid approaches combining CP-SAT’s optimality-seeking with the RL baseline’s pattern recognition, for example using CP-SAT to generate diverse training data or using learned policies to warm-start branch-and-bound search. Transfer learning also deserves investigation: can models pretrained on shorter sequences accelerate longer ones, and do they transfer across sequences with similar hydrophobic patterns? Hybrid pipelines that warm-start CP-SAT with H-core skeletons drawn from CPSP-tools, or that use CP-SAT to extend coverage beyond the precomputed H-core database (as required for sequences such as 3d8 and for non-cubic lattices), are a natural next step.

Code Availability

The software, code, and data used in this study are available in public GitHub repositories and Github Gists:

- **CP-SAT Implementation:**
<https://github.com/pranav-ramanathan/HP-CSP>
- **Attention-based DQN Baseline Implementation:**
<https://github.com/AnonymousAuthor117/Protein-Structure-Prediction-in-the-3D-HP-Model-Using-Deep-Reinforcement-Learning>
- **CPSP-tools reference runs:**
[scripts/run_cpssp_reference.py](#), [scripts/analyze_cpssp_reference.py](#),
[results/cpssp_reference/](#)
- **Image Generation Prompts:**
<https://gist.github.com/pranav-ramanathan/f5a4f8effa07861ef452e802132e2dc8>

References

- [1] Ken A. Dill. Theory for the folding and stability of globular proteins. *Biochemistry*, 24(6):1501–1509, 1985.
- [2] Kit Fun Lau and Ken A. Dill. A lattice statistical mechanics model of the conformational and sequence spaces of proteins. *Macromolecules*, 22(10):3986–3997, 1989.
- [3] Pierluigi Crescenzi, Deborah Goldman, Christos Papadimitriou, Antonio Piccolboni, and Mihalis Yannakakis. On the complexity of protein folding. *Journal of Computational Biology*, 5(3):423–465, 1998.
- [4] Ron Unger and John Moult. Genetic algorithms for protein folding simulations. *Journal of Molecular Biology*, 231(1):75–81, 1993.

- [5] Natalio Krasnogor, B. P. Blackburne, Edmund K. Burke, and Jonathan D. Hirst. Multimeme algorithms for protein structure prediction. In Juan Julián Merelo Guervós, Panagiotis Adamidis, Hans-Georg Beyer, Hans-Paul Schwefel, and José Luis Fernández-Villacañás, editors, *Parallel Problem Solving from Nature – PPSN VII*, volume 2439 of *Lecture Notes in Computer Science*, pages 769–778. Springer, Berlin, Heidelberg, 2002. doi:10.1007/3-540-45712-7_74. URL <https://dl.acm.org/doi/10.5555/645826.669429>.
- [6] Neal Lesh, Michael Mitzenmacher, and Sue Whitesides. A complete and effective move set for simplified protein folding. In *Proceedings of the 7th Annual International Conference on Computational Molecular Biology (RECOMB’03)*, pages 188–195, 2003.
- [7] Thomas Stützle and Holger H. Hoos. Max-min ant system. *Future Generation Computer Systems*, 16(8):889–914, 2000.
- [8] Roman Barták. Theory and practice of constraint propagation. In *Proceedings of the 3rd Workshop on Constraint Programming for Decision and Control*, pages 7–14, 2001.
- [9] Martin Mann, Sebastian Will, and Rolf Backofen. Csp-web-tools: a server for 3d lattice protein studies. *Bioinformatics*, 25(5):676–677, 2009.
- [10] Laurent Perron, Frédéric Didier, and Vianney Furnon. The cp-sat-lp solver. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, 2023.
- [11] Google. Or-tools: Google optimization tools. <https://developers.google.com/optimization>, 2024.
- [12] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming*, pages 417–431, 1998.
- [13] Peizheng Liu and Hitoshi Iba. Enhancing reinforcement learning in 3-dimensional hydrophobic-polar protein folding model with attention-based layers. *arXiv preprint arXiv:2504.15634*, 2025. URL <https://arxiv.org/abs/2504.15634>.
- [14] Kaiming Yang et al. Applying deep reinforcement learning to the hp model for protein structure prediction. *Physica A: Statistical Mechanics and Its Applications*, 608:128395, 2022.